

End-to-End CPU Efficiency of a Binary GGM Vision Transformer

Reviewer-response insert: FP32-versus-binary latency, throughput, persistent state, process memory, CPU attribution, batch scaling, and projection/full-precision accounting

Reviewer question. “Provide actual efficiency measurements (latency, throughput, memory, energy, or bit-ops) for a representative ViT, accounting for projection expansion and all full-precision modules. This is the decisive item.”

Response. For a complete 12-block, width-1024, 224×224 Vision Transformer evaluated on CPU at batch 2, the binary GGM implementation is **2.106 $\times$ faster**, reduces latency by **52.5%**, increases throughput from **1.153** to **2.428 images/s**, uses **22.54 $\times$ less registered persistent tensor state**, produces a **22.51 $\times$ smaller serialized state dictionary**, and reduces peak process RSS by **51.8%** relative to the FP32 baseline; the complete forward path includes the GGM/binary transformations executed by the model together with patch embedding, attention, normalization, residual operations, feed-forward activations, and the classifier.

Evaluated topology and implementation boundary. Both implementations use a 224×224 RGB input, 16×16 patches, model width $d = 1024$, 12 Transformer blocks, four attention heads, feed-forward ratio 4, and 1,000 output classes. The comparison is between a standard FP32 Vision Transformer and the GGM-based binary implementation.

Component	FP32 baseline	Binary GGM runtime treatment
Patch embedding	FP32 convolution	Retained CPU convolution; included in end-to-end timing
Transformer body	Dense FP32 linear algebra	Binary/GGM implementation with compressed registered state
Attention	Batched matrix products and softmax	Retained batched matrix products and softmax; fully measured
Activations / norms	FP32 GELU, elementwise operations, and LayerNorm	Retained elementwise operations and LayerNorm
Persistent state	Registered FP32 parameters	Small parameter set plus registered GGM/binary buffers
Temporary state	CPU operator intermediates	Projection, conversion, copies, attention, and other intermediates included
Classifier	FP32 dense output layer	Retained floating-point output boundary

Projection accounting. The benchmark measures the complete model call, so all projection, conversion, packing, copying, and binary operations actually executed by the current GGM implementation are charged to the reported latency and process-memory results.

Headline protocol. AMD EPYC 7F72 CPU node; Linux 5.14 x86-64; PyTorch 2.4.0+cu121; 48 physical and 48 logical CPUs available; eight PyTorch intra-op threads; 48 inter-op threads; MKLDNN enabled; random FP32 inputs; randomly initialized parameters; inference mode; ten warm-ups; and `torch.utils.benchmark.Timer.blocked_autorange` for timing.

Model	Latency / batch	Latency / image	Throughput	Timing IQR
FP32 Baseline	1,734.74 ms	867.37 ms	1.153 img/s	12.14 ms
Binary GGM	823.59 ms	411.79 ms	2.428 img/s	4.82 ms
Relative result	52.5% lower	52.5% lower	2.106 $\times$	—

Persistent tensor-state and serialized-state accounting.

Model	Parameters	Buffers	Persistent state	Serialized state
FP32 Baseline	583.88 MiB	0.00 MiB	583.88 MiB	583.93 MiB
Binary GGM	7.88 MiB	18.02 MiB	25.90 MiB	25.94 MiB
Compression	—	—	22.54 $\times$	22.51 $\times$
Reduction	—	—	95.6%	95.6%

The binary GGM model replaces most FP32 parameter storage with a much smaller combination of parameters and buffers. Registered buffers form most of the remaining persistent state and are consistent with implementation-specific transformed or binary representations. A semantic accounting of each buffer requires direct inspection of the model implementation.

Whole-process memory accounting.

Model	Baseline RSS	Model/input RSS increase	Peak process RSS	Inference peak increase
FP32 Baseline	418.54 MiB	585.80 MiB	1.04 GiB	59.94 MiB
Binary GGM	419.29 MiB	40.80 MiB	513.23 MiB	53.14 MiB
Binary change	$\approx 0\%$	93.0% lower	51.8% lower	11.3% lower

Peak process RSS is the highest resident-memory footprint reached by the complete benchmark process. It includes Python, imported libraries, PyTorch and MKLDNN runtime state, model parameters and buffers, inputs, temporary activations, operator workspaces, allocator caches, and shared-library pages attributed to the process.

Reviewer question. “Which Transformer modules are quantized, and how much do retained full-precision components contribute? Does the gain survive realistic projection and FP accounting?”

Response. In each Transformer block, binary GGM replaces the six dense linear maps—the Q , K , V , and attention-output projections together with the two feed-forward projections—with Fast-JL activation projection and packing followed by packed binary linear operations, while patch embedding, attention score/value matrix multiplications, softmax, LayerNorm, GELU, residual additions, positional and class embeddings, the final normalization, and the classifier remain full precision; all of these retained operations and all runtime projection/packing costs are included in the end-to-end measurement, and the binary implementation still reduces the profiled linear/GEMM share from **88.77%** to **0.29%** and remains faster at every measured batch from 1 to 32.

Code-level quantization and full-precision map. The binary portion of each encoder block consists of six packed weight matrices: `W_q_packed`, `W_k_packed`, `W_v_packed`, `W_proj_packed`, `W_ff1_packed`, and `W_ff2_packed`. The corresponding floating-point weights are created only during model initialization, immediately transformed by `prepare_weight_fast_jl`, registered as packed buffers, and then discarded; the original floating-point versions are therefore not retained in the model state.

During every forward pass, activations are projected and packed before the binary operations. Each block calls the embedding projection and bit-packing module four times: once for the shared $Q/K/V$ input, once before the attention output projection, once before the first feed-forward projection, and once before the second feed-forward projection. Thus, the 12-block model performs 48 activation projection/packing calls and 72 packed binary linear operations per forward pass. The Q , K , and V projections share one packed activation, avoiding three separate projections of the same input.

The following components remain full precision and are included in the reported latency and memory measurements:

- the convolutional patch embedding;
- the class token and positional embedding;
- both LayerNorm operations in every encoder block;
- attention score computation QK^T ;
- softmax and the attention-value product;
- GELU in the feed-forward block;
- residual additions;
- the final LayerNorm; and
- the final 1,000-class linear head.

The projection modules operate on only two input dimensions across the entire network. The Q , K , V , attention-output, and first feed-forward projections use inputs of width d_{model} , while the second feed-forward projection uses inputs of width $d_{\text{hidden}} = 4d_{\text{model}}$. Consequently, all Transformer layers can share two global Rademacher sign vectors: one for d_{model} and one for d_{hidden} . These vectors are generated once, registered as buffers, and reused by every corresponding projection module, rather than storing separate random signs for each layer or weight matrix.

Runtime component	Observed implementation evidence	Accounting status
Dense Transformer linear work	FP32 dominated by <code>aten::mm</code> ; binary path reduces linear/GEMM self time to 0.29%	Included
Attention	<code>aten::bmm</code> and <code>aten::_softmax</code> remain visible	Retained floating-point cost included
Normalization	<code>aten::native_layer_norm</code> remains visible	Retained floating-point cost included
Feed-forward activation	<code>aten::gelu</code> , <code>aten::mul</code> , and <code>aten::div</code> remain visible	Retained cost included
Patch embedding	<code>aten::mkldnn_convolution</code> remains visible	Retained floating-point cost included
Residual / elementwise work	<code>aten::add</code> and other elementwise operators remain visible	Included
Projection / conversion / packing	Reflected through copy, conversion, allocation, and binary-path runtime	Included end to end; not separately named
Classifier / dense boundary	Small <code>aten::addmm</code> contribution remains	Retained floating-point cost included

Measured CPU-time attribution. A five-forward `torch.profiler` trace shows that the FP32 baseline is dominated by dense linear/GEMM work, whereas binary GGM shifts the bottleneck to elementwise computation, tensor movement, and retained attention. Self CPU time is an attribution tool rather than an uninstrumented latency measurement and may be perturbed by profiler overhead.

Component	FP32 self CPU	FP32 share	Binary GGM self CPU	Binary GGM share
Linear / GEMM	7,962.56 ms	88.77%	4.05 ms	0.29%
Attention	339.67 ms	3.79%	336.92 ms	23.73%
Layout / copy	311.28 ms	3.47%	450.20 ms	31.71%
Activation / elementwise	234.74 ms	2.62%	514.01 ms	36.20%
Normalization	63.17 ms	0.70%	58.92 ms	4.15%
Patch embedding convolution	41.02 ms	0.46%	37.88 ms	2.67%
Other	17.45 ms	0.19%	17.85 ms	1.26%

Binary GGM operator hotspots.

Operator	Self CPU	Calls	Likely role
<code>aten::copy_</code>	440.05 ms	1,030	Tensor movement, assignment, conversion, or materialization
<code>aten::bmm</code>	311.14 ms	120	Batched attention matrix products
<code>aten::mul</code>	163.18 ms	300	Elementwise scaling or gating
<code>aten::div</code>	151.21 ms	360	Elementwise normalization or scaling
<code>aten::gelu</code>	148.72 ms	60	Feed-forward activation
<code>aten::native_layer_norm</code>	58.35 ms	125	Layer normalization
<code>aten::add</code>	50.89 ms	125	Residual or elementwise addition
<code>aten::mkldnn_convolution</code>	37.72 ms	5	Patch embedding
<code>aten::_softmax</code>	25.53 ms	60	Attention softmax
<code>aten::empty</code>	7.24 ms	1,230	Temporary tensor allocation
<code>aten::to_copy</code>	4.88 ms	780	Dtype or layout conversion wrapper
<code>aten::empty_strided</code>	3.63 ms	780	Strided temporary allocation
<code>aten::reshape</code>	2.37 ms	480	Shape or layout handling
<code>aten::addmm</code>	2.06 ms	5	Retained dense output boundary
<code>aten::matmul</code>	1.97 ms	120	Remaining matrix operations

Contribution of retained full-precision work. In the binary GGM profile, retained attention contributes 23.73% of self CPU time, normalization 4.15%, patch embedding 2.67%, and standard activation/elementwise work 36.20%. Layout and copy operations contribute another 31.71%, reflecting practical projection/conversion/materialization overhead. Thus, the reported speedup already includes the substantial cost of non-binary components and implementation overhead.

Optimization implication. The FP32 baseline is dense-compute dominated, whereas the current binary GGM implementation is memory-movement and elementwise-operation limited. The main optimization targets are repeated conversion/materialization, copy-heavy packing or assignment, temporary allocation, retained attention, and the lack of a fused multi-threaded GGM kernel.

Multi-threading boundary. The current GGM layer is **not fully optimized for CPU multi-thread execution**. Although eight PyTorch intra-op threads were configured, the custom GGM path does not yet exploit the available cores as efficiently as mature MKLDNN/BLAS FP32 kernels. The result is therefore a valid measurement of the current implementation, but not a final optimized many-core CPU result.

Batch-size sweep. Binary GGM remains faster at every measured batch from 1 to 32, although its advantage decreases as the batch grows.

Batch	FP32 latency	Binary latency	Speedup	FP32 throughput	Binary throughput
1	905.12 ms	400.86 ms	2.258×	1.105 img/s	2.495 img/s
2	1,724.58 ms	825.44 ms	2.089×	1.160 img/s	2.423 img/s
4	3,330.78 ms	1,741.89 ms	1.912×	1.201 img/s	2.296 img/s
8	6,683.21 ms	3,504.00 ms	1.907×	1.197 img/s	2.283 img/s
16	13,584.17 ms	8,162.40 ms	1.664×	1.178 img/s	1.960 img/s
32	27,483.53 ms	17,632.64 ms	1.559×	1.164 img/s	1.815 img/s

Does the gain survive realistic accounting? Yes, for the measured operating region. The complete binary model remains 2.258×, 2.089×, 1.912×, 1.907×, 1.664×, and 1.559× faster at batches 1, 2, 4, 8, 16, and 32, respectively. These measurements include retained floating-point attention, normalization, patch embedding, activations, residual operations, projection/conversion overhead, and process-level temporary memory.

Reproducibility.

For reproducibility, simply run `cpu_benchmark.py` from the terminal.